COMPLETE DEVELOPER SETUP GUIDE · 2026

Run OpenClaw Anywhere You Want

The complete guide to setting up OpenClaw on a Mac mini (local, persistent, boot-on-startup) and deploying to fly.io (cloud, always-on, accessible from anywhere).

- → Mac mini local setup
- → Telegram integration
- → Boot persistence (launchd)
- → Private deployments
- → fly.io cloud deployment
- → Secrets & API keys
- → Model provider config
- → Full troubleshooting guide

Contents

GETTING STARTED

What is OpenClaw? 3

What you'll need 3

Mac mini vs fly.io — when to use which 3

PART 1 — MAC MINI SETUP

Prerequisites: Node.js 24 4

Installing OpenClaw 4

Running the onboarding wizard 4

Configuration: openclaw.json 5

Secrets: the .env file 5

Running as a launchd service (boot persistence) 5

Connecting Telegram 6

Verifying everything works 6

Mac mini troubleshooting 7

PART 2 — FLY.IO DEPLOYMENT

Why fly.io 8

Prerequisites: flyctl 8

Creating the app and volume 8

Configuring fly.toml 9

Setting secrets 9

Deploying 10

Creating config via SSH 10

Connecting Telegram on fly.io 11

Updating and managing the deployment 11

fly.io troubleshooting 12

PART 3 — ADVANCED CONFIGURATION

Model providers and fallbacks 14

Private fly.io deployment (no public IP) 15

Group chats in Telegram 15

In-chat commands reference 16

Quick reference cheatsheet 17

INTRODUCTION

What is OpenClaw?

OpenClaw is an open-source AI agent gateway that connects LLMs to your tools, files, and messaging apps. Think of it as a personal AI assistant that runs on your infrastructure, responds in Telegram/Slack/Discord, can browse the web, run shell commands, read your files, and execute code — using any model provider you choose.

OpenClaw crossed 100,000 GitHub stars in early 2026. It supports Anthropic Claude, OpenAI GPT, Google Gemini, DeepSeek, and local models via Ollama. You pay only for API calls — the software is MIT-licensed and free.

> ℹ️ **OpenClaw vs Claude Code**
> OpenClaw is a persistent AI gateway designed for ongoing use across messaging platforms. Claude Code is a terminal-based coding assistant. They're complementary tools, not competitors.

What You'll Need

Requirement	Mac Mini Setup	fly.io Setup
Node.js	v24+ (required)	Handled by Docker
flyctl CLI	Not needed	Required
Git	Optional	Required (to clone repo)
Model API key	At least one	At least one
Telegram bot token	Optional	Optional
fly.io account	Not needed	Required (free tier works)
Cost	$0 (local only)	~$10–15/month

Mac Mini vs fly.io — When to Use Which

Scenario	Use
You want OpenClaw always on, on your local network	Mac mini
You want to access your agent from anywhere (phone, work, travel)	fly.io
You don't want to pay monthly hosting costs	Mac mini
Your Mac mini sleeps or is turned off sometimes	fly.io
You want Telegram webhooks (faster than polling)	fly.io
Privacy-first, no data leaving your network	Mac mini

Many developers run both: Mac mini for local development/experimentation, fly.io for their always-on personal assistant.

PART 1

Mac Mini Setup

Set up OpenClaw to run locally on your Mac mini, starting automatically on boot via launchd.

Step 1: Install Node.js 24

OpenClaw requires Node.js 22.16+ and recommends Node.js 24. Check your current version first:

```
node --version
```

If you need to install or upgrade, use `nvm` (recommended) or download directly from nodejs.org:

```
# Using nvm (recommended)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bas
source ~/.zshrc
nvm install 24
nvm use 24
nvm alias default 24
```

⚠️ **Do not use bun in production**

The OpenClaw docs explicitly say bun is NOT recommended for running the gateway. Use Node.js.

Step 2: Install OpenClaw

The official installer is the simplest option:

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

Alternatively, install via npm:

```
npm install -g openclaw@latest
```

Verify the install worked:

```
openclaw --version
```

Step 3: Run the Onboarding Wizard

The wizard sets up your config and installs the background service in one go:

```
openclaw onboard --install-daemon
```

The `--install-daemon` flag is important — it installs the `launchd` user service so OpenClaw starts automatically at boot. Without it you only get an interactive session.

The wizard (about 2 minutes) will ask you to:

- Choose a model provider (Anthropic, OpenAI, Google, etc.)
- Enter your API key
- Configure the gateway

> ✅ **After onboarding**
> Run `openclaw gateway status` to confirm the gateway is running. You should see "Runtime: running, RPC probe: ok".

Step 4: Configuration File

OpenClaw's config lives at `~/.openclaw/openclaw.json`. It uses JSON5 format — comments and trailing commas are allowed. Here's a minimal working config:

```
{
  agent: {
    model: "anthropic/claude-opus-4-6",
  },
}
```

A fuller config with fallback models and Telegram enabled:

```
{
  agents: {
    defaults: {
      model: {
        primary: "anthropic/claude-opus-4-6",
        fallbacks: ["anthropic/claude-sonnet-4-6", "openai/gpt-4o"]
      },
    },
  },
  channels: {
    telegram: {
      enabled: true,
    },
  },
  gateway: {
    mode: "local",
  },
}
```

Step 5: The .env File (Critical for Services)

When OpenClaw runs as a background launchd service, it may not inherit your shell environment. This means API keys set in your `.zshrc` won't be available. **Always put secrets in the dedicated env file:**

```
# File: ~/.openclaw/.env
ANTHROPIC_API_KEY=sk-ant- ...
OPENAI_API_KEY=sk- ...
TELEGRAM_BOT_TOKEN=123456789:ABCDEF ...
OPENCLAW_GATEWAY_TOKEN=# generate with: openssl rand -hex 32
```

🚫 **Do not put the state directory in iCloud**

Never place `~/.openclaw` inside `~/Library/Mobile Documents/` or `~/Library/CloudStorage/`. This causes database corruption. OpenClaw will warn you via `openclaw doctor` if you do this. Use the default `~/.openclaw` path.

Step 6: Boot Persistence with launchd

If you used `openclaw onboard --install-daemon`, the service is already installed. To verify or reinstall manually:

```
# Install the service
openclaw gateway install

# Check the service is running
openclaw gateway status

# Restart the service
openclaw gateway restart

# Force reinstall after issues
openclaw gateway install --force
openclaw gateway restart
```

The service label is `ai.openclaw.gateway`. You can also control it directly via launchctl:

```
# Restart
launchctl kickstart -k gui/$UID/ai.openclaw.gateway

# Stop
launchctl bootout gui/$UID/ai.openclaw.gateway
```

Check the dashboard in your browser (runs at `http://127.0.0.1:18789/`):

```
openclaw dashboard
```

Step 7: Connecting Telegram

1. **Create a bot via BotFather**

   Open Telegram and message **@BotFather**. Send `/newbot` and follow the prompts to choose a name and username. Copy the bot token — it looks like `123456789:ABCDefGhIJKlmNoPQRsTUVwxyZ`.

2. **Add the token to your .env file**

   Add `TELEGRAM_BOT_TOKEN=your-token-here` to `~/.openclaw/.env`. Restart the gateway after saving.

3. **Restart and message your bot**

   Run `openclaw gateway restart`, then find your bot in Telegram and send it a message. It will respond with a 4-digit pairing code.

4. **Approve the pairing request**

   On your Mac, run the command below with your pairing code:

```
# List pending pairing requests
openclaw pairing list telegram

# Approve with the code from your bot's message
openclaw pairing approve telegram <CODE>
```

> ⚠️ **Pairing codes expire after 1 hour**
> If the code expires, just send another message to your bot to get a new one. A maximum of 3 pending requests are held at a time.

Step 8: Verify Everything Works

Run these diagnostic commands in order. They'll surface most common issues:

```
# Full status check
openclaw status --all

# Gateway health
```

```
openclaw gateway status

# Channel connectivity
openclaw channels status --probe

# Doctor (checks for common config issues)
openclaw doctor

# Live logs
openclaw logs --follow
```

If everything is working, your Telegram bot will respond to messages. Try asking it: *"What's today's date?"* — if it answers, you're live.

> ✅ **You're done with the Mac mini setup**
> OpenClaw is now running locally, will start automatically on boot, and is accessible via Telegram. Skip to Part 3 for advanced configuration, or read on for fly.io deployment.

Mac Mini Troubleshooting

`"Onboarding stuck at Wake up my friend"`

Gateway is unreachable. Run: `openclaw gateway restart` then `openclaw status` and `openclaw doctor` to identify the issue.

`"Gateway start blocked: set gateway.mode=local"`

Add `gateway.mode: "local"` to your `~/.openclaw/openclaw.json`, or run `openclaw configure`.

`"refusing to bind gateway ... without auth"`

You're trying to bind to a non-loopback address without a gateway token. Add to your config: `gateway: { bind: "lan", auth: { mode: "token", token: "your-token" } }`. Generate a token: `openssl rand -hex 32`

`"another gateway instance is already listening" / EADDRINUSE`

Port 18789 is already in use. Check: `lsof -i :18789`. Kill the conflicting process or restart the service: `openclaw gateway restart`

`"No credentials found for profile"`

The launchd service isn't loading your API keys. Put them in `~/.openclaw/.env` (not `.zshrc` — services don't load shell profiles). Restart after saving.

`"HTTP 429 rate_limit_error"`

You've hit your API rate limit. Either wait for the window to reset, or add fallback models to your config (see Part 3 — Model Fallbacks).

`Bot not responding in Telegram groups`

Two things to check: (1) Disable Telegram privacy mode via @BotFather → /setprivacy, then remove and re-add the bot to the group. (2) Add the group ID to your config under `channels.telegram.groups`.

Diagnostic command sequence

Run these in order when something isn't working:

```
openclaw status
openclaw status --all       # safe to paste for debugging
openclaw gateway status
openclaw status --deep      # probes the running gateway
openclaw logs --follow
openclaw doctor
openclaw channels status --probe
openclaw health --json
```

Updating OpenClaw

```
openclaw update                    # latest stable
openclaw update --channel beta     # latest beta
openclaw update --dry-run          # preview without applying

# Always run after updating:
openclaw doctor
openclaw gateway restart
openclaw health
```

PART 2

fly.io Deployment

Deploy OpenClaw to fly.io for a cloud-hosted, always-on agent accessible from anywhere in the world.

Why fly.io?

fly.io is the recommended cloud platform for OpenClaw because it ships with a native `fly.toml` config in the repo. It offers persistent volumes (so your state survives restarts), global regions, and a free tier that covers light usage. Expect ~$10–15/month for a always-on 2GB instance.

Step 1: Install flyctl

```
# macOS / Linux
curl -L https://fly.io/install.sh | sh

# macOS with Homebrew
brew install flyctl

# Verify
fly version
```

Sign up and authenticate:

```
fly auth signup    # new account
fly auth login     # existing account
```

Step 2: Clone the Repository

fly.io builds from the `Dockerfile` in the repo, so you need a local copy:

```
git clone https://github.com/openclaw/openclaw.git
```

```
cd openclaw
```

Step 3: Create the App and Persistent Volume

```
# Create your app (choose a unique name)
fly apps create my-openclaw

# Create a 1GB persistent volume
# Replace 'iad' with your nearest region:
# lhr = London, iad = Virginia, sjc = San Jose
# fra = Frankfurt, syd = Sydney, nrt = Tokyo
fly volumes create openclaw_data --size 1 --region iad
```

ℹ️ **The volume is critical**

All your config, state, and agent memory lives on this volume at `/data`. Without it, everything resets on every deploy.

Step 4: Configure fly.toml

The repo includes a `fly.toml`. Edit it to match your app name and region. Here's the complete recommended config with explanations:

```
# fly.toml — edit these values:
app = "my-openclaw"          # your app name from Step 3
primary_region = "iad"       # your region from Step 3

[build]
  dockerfile = "Dockerfile"

[env]
  NODE_ENV = "production"
  OPENCLAW_PREFER_PNPM = "1"
  OPENCLAW_STATE_DIR = "/data"      # REQUIRED — saves state to volume
  NODE_OPTIONS = "--max-old-space-size=1536"

[processes]
  app = "node dist/index.js gateway --allow-unconfigured --port 3000 --bind la

[http_service]
  internal_port = 3000            # must match --port above
  force_https = true
  auto_stop_machines = false      # IMPORTANT: keep running for Telegram polli
  auto_start_machines = true
  min_machines_running = 1
  processes = ["app"]

[[vm]]
  size = "shared-cpu-2x"
  memory = "2048mb"              # 512MB is too small — use at least 2GB

[mounts]
  source = "openclaw_data"
  destination = "/data"
```

🚫 **Do NOT set auto_stop_machines = true**

If the machine stops, OpenClaw loses its Telegram/Discord polling connections. Telegram will stop delivering messages. Always set this to `false`.

Step 5: Set Secrets

Never put API keys in `fly.toml` — they'd be visible in git. Use Fly secrets instead:

```
# REQUIRED: Gateway auth token (mandatory for non-loopback binding)
fly secrets set OPENCLAW_GATEWAY_TOKEN=$(openssl rand -hex 32)

# Model providers — set at least one
fly secrets set ANTHROPIC_API_KEY=sk-ant-...
fly secrets set OPENAI_API_KEY=sk-...
fly secrets set GOOGLE_API_KEY=...

# Messaging channels (optional)
fly secrets set TELEGRAM_BOT_TOKEN=123456:ABCDEF...
fly secrets set DISCORD_BOT_TOKEN=MTQ...
fly secrets set SLACK_BOT_TOKEN=xoxb-...
```

Verify secrets were saved:

```
fly secrets list
```

Step 6: Deploy

```
fly deploy
```

The first deploy builds the Docker image — takes 2–3 minutes. Subsequent deploys are faster. Watch for a successful health check in the output.

After deploy, check it's running:

```
fly status
fly logs
```

A healthy startup looks like this in the logs:

```
[gateway] listening on ws://0.0.0.0:3000 (PID 17)
[telegram] logged in as @yourbotname
```

Step 7: Create Your Config via SSH

The `--allow-unconfigured` flag in `fly.toml` lets the gateway start without a config, so we can create it after the first deploy:

```
# SSH into your running machine
fly ssh console
```

Inside the container, create `/data/openclaw.json`:

```
cat > /data/openclaw.json << 'EOF'
{
  "agents": {
    "defaults": {
      "model": {
        "primary": "anthropic/claude-opus-4-6",
        "fallbacks": ["anthropic/claude-sonnet-4-6", "openai/gpt-4o"]
```

```
      },
      "maxConcurrent": 4
    },
    "list": [{ "id": "main", "default": true }]
  },
  "channels": {
    "telegram": { "enabled": true }
  },
  "gateway": {
    "mode": "local",
    "bind": "auto"
  }
}
EOF
exit
```

> ℹ️ **Channel tokens come from secrets, not config**
> You don't need to put `botToken` in `openclaw.json` on fly.io — the `TELEGRAM_BOT_TOKEN` secret you set in Step 5 is picked up automatically from the environment.

Restart the machine to apply the new config:

```
fly machine list                        # get your machine ID
fly machine restart <machine-id>
```

Then check the logs to confirm it started cleanly:

```
fly logs
```

Step 8: Connect Telegram on fly.io

The bot token is already set via Fly secrets. Now pair your Telegram account:

1. **Message your bot in Telegram**

   Find your bot by the username you set in BotFather and send any message. The bot will respond with a 4-character pairing code.

2. **Approve the pairing via SSH**

   SSH into your fly.io machine and approve the code:

```
fly ssh console

# Inside the container:
openclaw pairing list telegram
openclaw pairing approve telegram <CODE>
exit
```

Send another message to your bot — it should now respond with the AI model of your choice.

Updating fly.io Deployments

```
# Pull latest source, then redeploy
git pull
fly deploy

# Check health after deploy
fly status
fly logs
```

⚠️ **Machine command resets after deploy**

If you've manually updated the machine command with `fly machine update`, it will be overwritten on the next `fly deploy`. Always make changes in `fly.toml`, not with manual machine commands.

Accessing the Control Dashboard

```
# Open the web dashboard (authenticate with OPENCLAW_GATEWAY_TOKEN)
fly open

# Or visit directly:
# https://my-openclaw.fly.dev/
```

Use the value you set for `OPENCLAW_GATEWAY_TOKEN` to authenticate. You can manage agents, review session history, and adjust config from the dashboard.

fly.io Cost Breakdown

Resource	Config	Approx. Cost/Month
VM	shared-cpu-2x, 2GB RAM	~$10–12
Volume	1GB persistent storage	~$0.15
Bandwidth	160GB included free	$0 (typical usage)
Total		**~$10–15/month**

fly.io Troubleshooting

`"App is not listening on expected address"`

The gateway is binding to `127.0.0.1` instead of `0.0.0.0`, so Fly's proxy can't reach it. Ensure your process command in `fly.toml` includes `--bind lan`.

`Health checks failing / connection refused on deploy`

The `internal_port` in `[http_service]` must match the `--port` value in your process command. Both should be `3000`.

`Machine silently restarts / OOM (SIGABRT, v8 allocation errors)`

The machine is running out of memory. 512MB is too small. Set `memory = "2048mb"` in `[[vm]]` in `fly.toml` and redeploy. Or update immediately without redeploying:

```
fly machines list
fly machine update <id> --vm-memory 2048 -y
```

`"Gateway Lock" — refuses to start with "already running"`

A PID lock file from a previous run is stuck on the volume. Remove it:

```
fly ssh console --command "rm -f /data/gateway.*.lock"
fly machine restart <machine-id>
```

`Config not being read / changes not taking effect`

Verify the config exists on the volume (not the container filesystem):

```
fly ssh console --command "cat /data/openclaw.json"
```

`State not persisting after restart`

The state directory is writing to the container filesystem instead of the mounted volume. Ensure `OPENCLAW_STATE_DIR=/data` is in the `[env]` block of `fly.toml` and redeploy.

`Cannot write config via fly ssh "pipe issue"`

The `-C` flag doesn't support shell redirection. Use tee instead:

```
echo '{"your":"config"}' | fly ssh console -C "tee /data/openclaw.json"
```

Or use sftp: `fly sftp shell` then `put /local/config.json /data/openclaw.json`

`Telegram bot stops responding after fly deploy`

The machine restarted and lost the pairing. You'll need to re-pair: message the bot, get a new code, then `fly ssh console` → `openclaw pairing approve telegram <CODE>`

PART 3

Advanced Configuration

Model fallbacks, private deployments, group chats, and performance tuning.

Model Providers and Fallbacks

OpenClaw supports any combination of providers. Configure fallbacks so your agent keeps working if one provider hits a rate limit or outage:

```
{
  agents: {
    defaults: {
      model: {
        primary: "anthropic/claude-opus-4-6",
        fallbacks: [
          "anthropic/claude-sonnet-4-6",
          "openai/gpt-4o",
          "google/gemini-3.1-pro-preview"
        ]
      }
    }
  }
}
```

All supported API key environment variables:

```
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...
GOOGLE_API_KEY=...
OPENROUTER_API_KEY=sk-or-...     # access 100+ models via one API
BRAVE_API_KEY=...                # web search
PERPLEXITY_API_KEY=pplx-...      # web search (alternative)
ELEVENLABS_API_KEY=...           # voice responses
```

Quick model shortcuts you can use in chat:

Shortcut	Resolves to
`/model opus`	anthropic/claude-opus-4-6
`/model sonnet`	anthropic/claude-sonnet-4-6
`/model gpt`	openai/gpt-5.4
`/model gemini`	google/gemini-3.1-pro-preview

Performance Tuning for Low-Power Instances

If OpenClaw starts slowly on your fly.io instance or Mac mini, enable the Node.js compile cache:

```
# Add to ~/.openclaw/.env (Mac) or fly.toml [env] (fly.io)
NODE_COMPILE_CACHE=/var/tmp/openclaw-compile-cache
```

On first run it compiles and caches the bytecode. Every subsequent start is significantly faster.

Private fly.io Deployment (No Public IP)

If you don't need inbound webhooks and want to keep your deployment off internet scanners, deploy with the private config:

```
# Deploy with no public IP (uses fly.private.toml from the repo)
fly deploy -c fly.private.toml
```

Access a private deployment:

```
# Option 1: Local proxy (simplest)
fly proxy 3000:3000 -a my-openclaw
# Then open: http://localhost:3000

# Option 2: SSH console
fly ssh console -a my-openclaw

# Option 3: WireGuard (access from any device)
fly wireguard create
# Import the config to your WireGuard client
```

> ℹ️ **Private mode and Telegram webhooks**
> Telegram webhooks require a public HTTPS URL. With a private deployment, use Telegram's long-polling mode (the default) instead. It works fine — Telegram just polls your bot for new messages rather than pushing them.

Setting Up Telegram Group Chats

To use your bot in a Telegram group:

1. **Disable bot privacy mode**

   Message @BotFather → `/setprivacy` → select your bot → choose `DISABLE`. Remove and re-add the bot to the group for this to take effect.

2. **Find your group's chat ID**

Forward a message from your group to @getidsbot in Telegram. Note the negative number (e.g. `-1001234567890`).

3 Add the group to your config

Edit `~/.openclaw/openclaw.json` (or the config on your fly.io volume):

```
{
  channels: {
    telegram: {
      enabled: true,
      groups: {
        "-1001234567890": {
          groupPolicy: "open",
          requireMention: false    // respond to all messages, not just @mentio
        }
      }
    }
  }
}
```

In-Chat Commands Reference

These commands work in any connected channel (Telegram, Slack, Discord, WhatsApp). Send them as standalone messages.

Command	What it does
`/status`	Current session status — model, token usage, cost
`/new` or `/reset`	Start a fresh conversation (clears context)
`/compact`	Summarise and compress the current context (saves tokens)
`/think minimal`	Fastest, cheapest responses (less reasoning)
`/think high`	Extended reasoning for complex tasks
`/think xhigh`	Maximum reasoning effort
`/verbose on`	Show tool calls and internal reasoning
`/usage full`	Show detailed token and cost tracking
`/model`	List available models / switch model
`/model opus`	Switch to Claude Opus 4.6
`/model sonnet`	Switch to Claude Sonnet 4.6
`/model gpt`	Switch to GPT-5.4
`/activation mention`	Only respond when mentioned (group chats)
`/activation always`	Respond to all messages (group chats)
`/restart`	Restart the gateway (owner only in groups)

CLI Commands Reference

```
# Gateway control
openclaw gateway start
openclaw gateway stop
```

```
openclaw gateway restart
openclaw gateway status
openclaw gateway install      # install as boot service

# Diagnostics
openclaw status --all
openclaw doctor
openclaw health --json
openclaw logs --follow

# Channels
openclaw channels status --probe
openclaw pairing list [channel]
openclaw pairing approve [channel] [code]

# Models
openclaw models list
openclaw models set anthropic/claude-opus-4-6

# Updates
openclaw update
openclaw update --dry-run
```

Quick Reference Cheatsheet

MAC MINI: KEY FILES

```
~/.openclaw/openclaw.json
~/.openclaw/.env
~/.openclaw/workspace/
```

FLY.IO: KEY FILES

```
/data/openclaw.json
fly.toml (in repo root)
fly secrets set KEY=value
```

MAC MINI: SERVICE CONTROL

```
openclaw gateway install
openclaw gateway restart
openclaw gateway status
openclaw doctor
```

FLY.IO: DEPLOYMENT

```
fly deploy
fly status
fly logs
fly ssh console
```

ENVIRONMENT VARIABLES

```
ANTHROPIC_API_KEY
OPENAI_API_KEY
TELEGRAM_BOT_TOKEN
OPENCLAW_GATEWAY_TOKEN
OPENCLAW_STATE_DIR=/data
```

COMMON FIX COMMANDS

```
openclaw gateway restart
openclaw doctor
openclaw logs --follow
openclaw status --all
```

FLY.IO: COMMON FIXES

```
fly machine restart <id>
fly ssh console
fly machine update <id> \
--vm-memory 2048 -y
rm -f /data/gateway.*.lock
```

IN-CHAT QUICK COMMANDS

```
/new — fresh session
/compact — save tokens
/model opus — switch model
/status — usage & cost
```

Questions? Issues not covered here?

OpenClaw official docs: **docs.openclaw.ai**

GitHub issues: **github.com/openclaw/openclaw/issues**

Guide by Maestro · getmetafix.com · Version 1.0 · March 2026